# Cheat Sheet: Building Unsupervised Learning Models

### Unsupervised learning models

| Model Name | Brief Description | Code Syntax |
|---|---|---|
| UMAP | UMAP (Uniform Manifold Approximation and Projection) is used for dimensionality reduction.<br>**Pros:** High performance, preserves global structure.<br>**Cons:** Sensitive to parameters.<br>**Applications:** Data visualization, feature extraction.<br>**Key hyperparameters:**<br>• **n_neighbors:** Controls the local neighborhood size (default = 15).<br>• **min_dist:** Controls the minimum distance between points in the embedded space (default = 0.1).<br>• **n_components:** The dimensionality of the embedding (default = 2). | `from umap.umap_ import UMAP`<br>`umap = UMAP(n_neighbors=15, min_dist=0.1, n_components=2)` |
| t-SNE | t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear dimensionality reduction technique.<br>**Pros:** Good for visualizing high-dimensional data.<br>**Cons:** Computationally expensive, prone to overfitting.<br>**Applications:** Data visualization, anomaly detection.<br>**Key hyperparameters:**<br>• **n_components:** The number of dimensions for the output (default = 2).<br>• **perplexity:** Balances attention between local and global aspects of the data (default = 30).<br>• **learning_rate:** Controls the step size during optimization (default = 200). | `from sklearn.manifold import TSNE`<br>`tsne = TSNE(n_components=2, perplexity=30, learning_rate=200)` |
| PCA | PCA (principal component analysis) is used for linear dimensionality reduction.<br>**Pros:** Easy to interpret, reduces noise.<br>**Cons:** Linear, may lose information in nonlinear data.<br>**Applications:** Feature extraction, compression.<br>**Key hyperparameters:**<br>• **n_components:** Number of principal components to retain (default = 2).<br>• **whiten:** Whether to scale the components (default = False).<br>• **svd_solver:** The algorithm to compute the components (default = 'auto'). | `from sklearn.decomposition import PCA`<br>`pca = PCA(n_components=2)` |
| DBSCAN | DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm.<br>**Pros:** Identifies outliers, does not require the number of clusters.<br>**Cons:** Difficult with varying density clusters.<br>**Applications:** Anomaly detection, spatial data clustering.<br>**Key hyperparameters:**<br>• **eps:** The maximum distance between two points to be considered neighbors (default = 0.5).<br>• **min_samples:** Minimum number of samples in a neighborhood to form a cluster (default = 5). | `from sklearn.cluster import DBSCAN`<br>`dbscan = DBSCAN(eps=0.5, min_samples=5)` |
| HDBSCAN | HDBSCAN (Hierarchical DBSCAN) improves on DBSCAN by handling varying density clusters.<br>**Pros:** Better handling of varying densities.<br>**Cons:** Can be slower than DBSCAN.<br>**Applications:** Large datasets, complex clustering problems.<br>**Key hyperparameters:**<br>• **min_cluster_size:** The minimum size of clusters (default = 5).<br>• **min_samples:** Minimum number of samples to form a cluster (default = 10). | `import hdbscan`<br>`clusterer = hdbscan.HDBSCAN(min_cluster_size=5)` |
| K-Means clustering | K-Means is a centroid-based clustering algorithm that groups data into k clusters.<br>**Pros:** Efficient, simple to implement.<br>**Cons:** Sensitive to initial cluster centroids.<br>**Applications:** Customer segmentation, pattern recognition.<br>**Key hyperparameters:**<br>• **n_clusters:** Number of clusters (default = 8).<br>• **init:** Method for initializing the centroids ('k-means++' or 'random', default = 'k-means++'). | `from sklearn.cluster import KMeans`<br>`kmeans = KMeans(n_clusters=3)` |

| Model Name | Brief Description | Code Syntax |
|---|---|---|
| | • **n_init:** Number of times the algorithm will run with different centroid seeds (default = 10). | |

### Associated fuctions used

| Method | Brief Description | Code Syntax |
|---|---|---|
| make_blobs | Generates isotropic Gaussian blobs for clustering. | `from sklearn.datasets import make_blobs`<br>`X, y = make_blobs(n_samples=100, centers=2, random_state=42)` |
| multivariate_normal | Generates samples from a multivariate normal distribution. | `from numpy.random import multivariate_normal`<br>`samples = multivariate_normal(mean=[0, 0], cov=[[1, 0], [0, 1]], size=100)` |
| plotly.express.scatter_3d | Creates a 3D scatter plot using Plotly Express. | `import plotly.express as px`<br>`fig = px.scatter_3d(df, x='x', y='y', z='z')`<br>`fig.show()` |
| geopandas.GeoDataFrame | Creates a GeoDataFrame from a Pandas DataFrame. | `import geopandas as gpd`<br>`gdf = gpd.GeoDataFrame(df, geometry='geometry')` |
| geopandas.to_crs | Transforms the coordinate reference system of a GeoDataFrame. | `gdf = gdf.to_crs(epsg=3857)` |
| contextily.add_basemap | Adds a basemap to a GeoDataFrame plot for context. | `import contextily as ctx`<br>`ax = gdf.plot(figsize=(10, 10))`<br>`ctx.add_basemap(ax)` |
| pca.explained_variance_ratio_ | Returns the proportion of variance explained by each principal component. | `from sklearn.decomposition import PCA`<br>`pca = PCA(n_components=2)`<br>`pca.fit(X)`<br>`variance_ratio = pca.explained_variance_ratio_` |

| Method | Brief Description | Code Syntax |
|---|---|---|
| | | |

## Author

Jeff Grossman
Abhishek Gagneja